

Article

An Explainable Deep Learning Pipeline for Malware Family Classification: GAF Image Encoding and API-Grounded LLM Interpretation

Youji Fukuta ^{1,*} , Yoshiaki Shiraishi ² , Masanori Hiroto ³  and Masami Mohri ¹ 

¹ Faculty of Informatics/Cyber Informatics Research Institute, Kindai University, Higashiosaka-shi 577-8502, Japan

² Graduate School of Engineering, Kobe University, Kobe-shi 657-8501, Japan

³ Faculty of Science and Engineering, Saga University, Saga-shi 840-8502, Japan

* Correspondence: yfukuta@info.kindai.ac.jp

Abstract

Signature-based malware detection is undermined by obfuscation and packing, motivating dynamic analysis of Application Programming Interface (API) call sequences. Existing image-based classifiers reach high accuracy but rarely explain why a sample belongs to a given family. In this paper, our goal is not to maximize classification accuracy but to demonstrate and characterize an explainable pipeline that both classifies and explains: API call sequences from the WinMET dataset are encoded as order-preserving Gramian Angular Field (GAF) images and classified with a ResNet-50, after which Grad-CAM activations are reverse-mapped to the contributing API calls, whose names, categories, arguments, and return values are passed to a single large language model (LLM) that generates a natural-language rationale. We evaluate classification and explanation jointly on ten malware families (16,771 samples) through three experiments: a GAF-versus-heatmap comparison under identical conditions, a per-family Grad-CAM faithfulness analysis, and a reference-free LLM-as-a-Judge assessment of interpretation quality. Consistent with this explanatory aim, GAF matched the heatmap on overall accuracy (about 0.80) while performing comparably on Macro-F1 (0.63 versus 0.61: higher on the single fixed split, with Welch's t -test $p = 0.015$, but statistically comparable under five-fold cross-validation); per-family faithfulness varied widely (0.08 to 0.80), and, under two independent LLM judges (GPT-4o and GPT-4.1), supplying argument-level context significantly reduced—rather than improved—the judged quality (best score from a zero-shot prompt without parameters: 13.34 of 15); a small human expert evaluation further indicated that the automated judges rewarded fluent but over-attributed rationales. This work contributes a feasible, fully containerized and reproducible framework for explainable malware family classification.



Academic Editor: Priyadarsi Nanda

Received: 27 June 2026

Revised: 3 August 2026

Accepted: 10 August 2026

Published: 14 August 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Keywords: malware family classification; deep learning; Gramian Angular Field; Grad-CAM; explainable AI; large language model; API call analysis; dynamic analysis; WinMET; cybersecurity

1. Introduction

Malicious software (malware) continues to proliferate at an extraordinary rate: the AV-TEST Institute registers more than 450,000 new malicious programs and potentially unwanted applications every day [1]. Signature-based static analysis, which has long been the backbone of commercial detection, is increasingly fragile in the face of obfuscation,

packing, and polymorphism, because such techniques alter the static representation of a sample while preserving its runtime behavior [2,3]. Dynamic analysis addresses this limitation by executing a sample in an instrumented sandbox and recording the sequence of Application Programming Interface (API) calls it issues. Because the API call sequence reflects the executed behavior directly, it has become an informative feature for malware family classification [3–5].

A productive line of work represents malware as images for CNN-based classification, beginning with byte-level malware images [2], which avoids hand-crafted feature engineering and reuses the mature toolbox of computer vision. For dynamic features, the API call sequence has likewise been encoded as an image and classified with convolutional neural networks (CNNs) [3,5]. In particular, the Gramian Angular Field (GAF), originally proposed by Wang and Oates for general time-series classification [6], encodes a one-dimensional sequence as a two-dimensional image while preserving temporal ordering, and its effectiveness for dynamic API call sequences has been reported [5]. A complementary approach transforms CAPEv2 sandbox JSON reports into structured heatmaps and classifies them with a ResNeXt-50 backbone [7]. In this work, image encoding is motivated primarily by explainability rather than by accuracy: representing the call sequence as an image provides the substrate on which Grad-CAM operates, allowing salient regions to be localized and reverse-mapped to the responsible API calls for LLM interpretation. Image encoding is thus integral to the explanation method rather than an accuracy optimization; prior work further indicates that it need not cost accuracy, as Zhang et al. [5] report accuracy gains from GAF imaging over the raw API call sequence in their setting; on WinMET, as Section 4 shows, we find the two comparable.

Despite these advances, most existing studies concentrate on improving classification accuracy and do not, to the best of our knowledge, automatically produce a human-readable account—at the semantic level of individual API calls—of *why* a sample is assigned to a given family. Explainable AI (XAI) techniques such as gradient-weighted class activation mapping (Grad-CAM) [8] have been applied to highlight salient regions [7,9], but a saliency map by itself is not an explanation an analyst can read. Moreover, the search for suitable training configurations in deep learning experiments still relies largely on manual effort, and, to the best of our knowledge, the domain-specific application of large language model (LLM) agents to automate this search in cybersecurity has not yet been explored [10]. Addressing these observations is central to the broader goal of developing novel deep learning approaches for cybersecurity that are not only accurate but also explainable and trustworthy to the analysts who must act on their output. We frame these observations as three gaps:

- **Gap 1:** To the best of our knowledge, no integrated pipeline transforms CAPEv2 API call sequences with GAF, reverse-maps Grad-CAM activations back to the contributing API calls, and produces a natural-language interpretation with a single LLM agent; the closest systems are positioned in Section 2.
- **Gap 2:** The rich API call parameters, return values, and category information available in modern execution traces have not been exploited for LLM interpretation. Existing studies use API call names only, whereas passing argument-level context to the LLM is, to our knowledge, novel.
- **Gap 3:** To the best of our knowledge, an LLM-agent loop that automatically optimizes malware-classification experiments has not been demonstrated; prior work on LLM-driven hyper-parameter optimization targets only generic tasks [10].

We frame the investigation around four research questions; the first three are addressed empirically in this paper, and the fourth is left to future work.

- **RQ1:** Does encoding WinMET API call sequences with an order-preserving GAF transform representative family-discriminative features more effectively than a frequency-only heatmap encoding (Gap 1)?
- **RQ2:** Does the Grad-CAM saliency underlying the reverse-mapping faithfully reflect the classifier across families, and does supplying the contributing calls' categories, arguments, and return values to the LLM yield a higher-quality rationale than a name-only baseline (Gaps 1–2)?
- **RQ3:** Can a single LLM agent generate a meaningful natural-language interpretation from the Grad-CAM-derived API context (Gaps 1–2)?
- **RQ4:** Can a domain-knowledge-guided LLM-agent loop reach competitive accuracy with fewer trials than grid search (Gap 3)?

In this paper, we propose an explainable deep learning pipeline that addresses these gaps by not only performing classification but also providing explanations for its predictions. The system takes a Windows malware execution trace, classifies its family, and produces a natural-language rationale grounded in the specific API calls responsible for the decision. We build on the recently released (2025) WinMET dataset [11,12], whose CAPEv2 provenance and inclusion of API parameters and return values make it well suited to argument-level interpretation. Throughout this paper, the term *pipeline* denotes the complete classification-and-explanation system; explanation generation is a constitutive part of it, not an optional add-on. Our objective is accordingly twofold. First, we demonstrate that such an integrated pipeline is feasible and reproducible. Second, we characterize its behavior along three axes: the effect of order preservation on classification, the faithfulness of the Grad-CAM reverse-mapping, and the value of argument-level context for interpretation. We do not aim to maximize classification accuracy or to benchmark against state-of-the-art classifiers, which we leave to the dedicated follow-up evaluation set out in Section 5.7. The significance of this work lies less in classification accuracy than in coupling family predictions with API-grounded explanations in a single reproducible framework and in characterizing, per family, where those explanations are faithful; the specific contributions are enumerated below.

The contributions of this paper are as follows:

- We design and implement an end-to-end explainable pipeline that couples GAF-based image classification with Grad-CAM reverse-mapping and single-agent LLM interpretation, so that every prediction is accompanied by an API-grounded, natural-language rationale.
- We incorporate WinMET API call parameters, return values, and categories as additional context for LLM interpretation, and we quantify their effect on explanation quality using an LLM-as-a-Judge protocol with two independent judges, finding that supplying such raw context does not improve and in fact significantly reduces judged quality.
- Under identical preprocessing and model conditions on WinMET, we experimentally compare GAF (order-preserving) against heatmap (frequency-only) transformations, isolating the effect of preserving temporal order on family classification (RQ1).
- We evaluate classification and explanation jointly with a reference-free protocol that requires no ground-truth rationales, and we characterize the per-family faithfulness of the Grad-CAM reverse-mapping, finding it to be strongly family-dependent (RQ2–RQ3).
- We provide a fully containerized, reproducible implementation, and we outline a domain-knowledge-guided LLM-agent loop for experiment optimization as a direction for future work.

The remainder of this paper is organized as follows: Section 2 reviews related work and positions our contribution. Section 3 presents the proposed pipeline, including the

dedicated explanation-generation stage. Section 4 describes the experimental setup and the evaluation of classification and explanation, including a human expert evaluation of the generated explanations. Section 5 discusses the findings, limitations, and future work, and Section 6 concludes.

2. Related Work

2.1. API Call-Sequence-Based Malware Classification

Dynamic API call sequences capture executed behavior and are widely used for malware detection and family classification. Recent deep learning methods extract semantic and temporal features from API sequences directly [3], and graph-based formulations model directed relationships between API calls to retain structural information [4]. A recurring challenge is class imbalance across malware families, which motivates data-augmentation strategies such as GAN-based oversampling [13], as well as loss reweighting. Our work likewise targets family classification from API sequences but differs in that classification is only one of two coupled objectives, the other being automated, human-readable explanation.

2.2. Encoding API Sequences as Images

Encoding time series as images enables the transfer of computer-vision architectures to sequence classification. Wang and Oates introduced the GAF and Markov Transition Field representations for this purpose [6]. For malware, API call sequences have been visualized and classified with CNNs, including GAF-based encodings of dynamic API sequences [5] and grayscale-image encodings combined with sequence features [3]. We adopt the GAF encoding of Zhang et al. [5] because it preserves the call order, and we contrast it with a frequency-only heatmap encoding in the spirit of El Rhayati et al. [7].

2.3. Explainability in Image-Based Malware Analysis

Grad-CAM [8] is the de facto saliency method for CNN-based malware classifiers. El Rhayati et al. [7] apply Grad-CAM over structured heatmaps generated from CAPEv2 reports, and comparative studies examine SHAP, LIME, and Grad-CAM for malware imagery [9]. These methods localize influential regions but stop short of producing a natural-language rationale at the level of named API calls. DECODE [14] couples visualization with LLM interpretation for multi-label behavior analysis but relies on a relatively complex, object-detection-based design. Like DECODE, our pipeline links salient regions back to the originating API calls and produces a natural-language interpretation; it differs in three deliberate ways: it uses a single LLM agent rather than a multi-agent reasoning module, it operates directly on GAF images and therefore requires no object-detection annotation, and it supplies the LLM with API categories, arguments, and return values rather than API names alone. To evaluate the generated explanations without reference texts, we later adopt an LLM-as-a-Judge protocol [15], in which a separate model scores each rationale.

2.4. LLM Agents for Experiment Optimization

LLM agents have recently been applied to hyper-parameter optimization for generic machine learning tasks, achieving competitive accuracy with fewer trials than grid search [10]. To the best of our knowledge, the cybersecurity-specific application of such an agent loop to malware-classification experiments has not been reported; we treat it as a forward-looking extension of our core pipeline.

2.5. Positioning

Table 1 summarizes how the proposed pipeline differs from representative prior work. Among the listed systems, ours combines an order-preserving image encoding, Grad-CAM XAI, and single-agent LLM interpretation on dynamic-analysis input within a fully containerized, reproducible environment; the domain-specific LLM-agent optimization is outlined as a future direction (Section 5.7) rather than claimed here. Two systems are closest to ours. El Rhayati et al. [7] share the CAPEv2-to-image-plus-Grad-CAM classification design but produce no natural-language explanation; relative to them, our differentiators are LLM interpretation, argument-level WinMET context, and the order-preserving GAF encoding. DECODE [14] is the closest system with respect to explanation, since it also maps XAI regions to API calls and interprets them with LLMs; relative to DECODE, our differentiators are a single-agent (rather than multi-agent) design, operation on GAF images without object-detection annotation, and the use of API arguments and return values as context. Because the prior studies use different datasets, we report method comparisons on WinMET rather than direct numerical comparisons of accuracy across datasets.

Table 1. Positioning of the proposed pipeline relative to representative prior work. “API→image” denotes the sequence-to-image encoding; “XAI” the explainability method; and “LLM interp.” whether a natural-language interpretation is produced. A dash (“–”) marks a capability the study does not provide; for this work this includes the LLM agent, which is outlined only for future work. The row for this work is set in bold to distinguish the proposed pipeline from the prior studies listed above it.

Study	API→Image	XAI	LLM Interp.	LLM Agent	Dyn. Analysis
GAF+CNN [5]	GAF	–	–	–	Yes
Zhang et al. [3]	Grayscale	–	–	–	Yes
El Rhayati et al. [7]	Heatmap	Grad-CAM	–	–	Yes
DECODE [14]	Obj. Det.	Bayesian	Multi	–	Yes
AgentHPO [10]	–	–	–	Generic	–
This work	GAF	Grad-CAM	Single	–	Yes

3. Proposed Explainable Pipeline

3.1. Dataset: WinMET

We use the Windows Malware Execution Traces (WinMET) dataset [11,12], generated with the CAPEv2 sandbox and the MALVADA framework [16] and released on Zenodo under a CC BY-NC 4.0 license. WinMET comprises 31,844 JSON execution traces, each recording, among other fields, API call names, categories, arguments, return values, the process tree, and network activity. Consensus family labels combining CAPE and AVClass are provided. We use the dataset for academic research purposes only, in accordance with its non-commercial license.

From the AVClass labels we select the ten most populated families, yielding 16,771 labeled samples. Table 2 lists the families and their sample counts; the distribution is imbalanced, which we address with class-weighted cross-entropy. The data are split into training, validation, and test partitions of 72%, 8%, and 20% in a stratified manner with a fixed random seed.

Table 2. Selected malware families from WinMET and their sample counts (10 families, 16,771 samples).

Family Count	Redline 4438	Vbclone 4023	Berbew 2794	Agenttesla 1201	Cosmu 899
Family Count	Taskun 856	Disabler 799	Amadey 763	Noon 530	Strab 468

3.2. Pipeline Overview

Figure 1 shows the proposed pipeline, a single classification-and-explanation system composed of seven steps; Table 3 lists them in detail. Steps 1–3 extract an API call sequence from the WinMET JSON and convert it into a fixed-length integer-index sequence. Step 4 applies the GAF transformation to obtain a 128×128 image. Step 5 classifies the image into one of ten families with a ResNet-50 [17]. Step 6 applies Grad-CAM [8] to identify the pixels that drove the prediction and reverse-maps them to the contributing API calls. Step 7 passes the predicted family, its confidence, and the contributing API calls—together with their categories, arguments, and return values—to an LLM, which produces a natural-language rationale. The system therefore emits two coupled outputs for every input: a family label and an API-grounded explanation of why that label was assigned. Steps 6 and 7 are constitutive stages of the pipeline rather than post hoc analyses. The step boxes in Figure 1 state the operative detail of each stage: the parent-process restriction and the collapsing of consecutive duplicates in Step 1, the 360-type vocabulary with index 0 reserved for padding in Step 2, the zero-padding to length 500 in Step 3, the piecewise aggregate approximation that reduces the 500 positions to the 128 image coordinates in Step 4, and, in Step 6, the rule by which the top-20 activated pixels are resolved into $N = 10$ contributing calls. Table 3 lists the input, the operation, and the output of every step, and the subsections that follow develop each stage in turn.

Explainable Malware Family Classification Pipeline

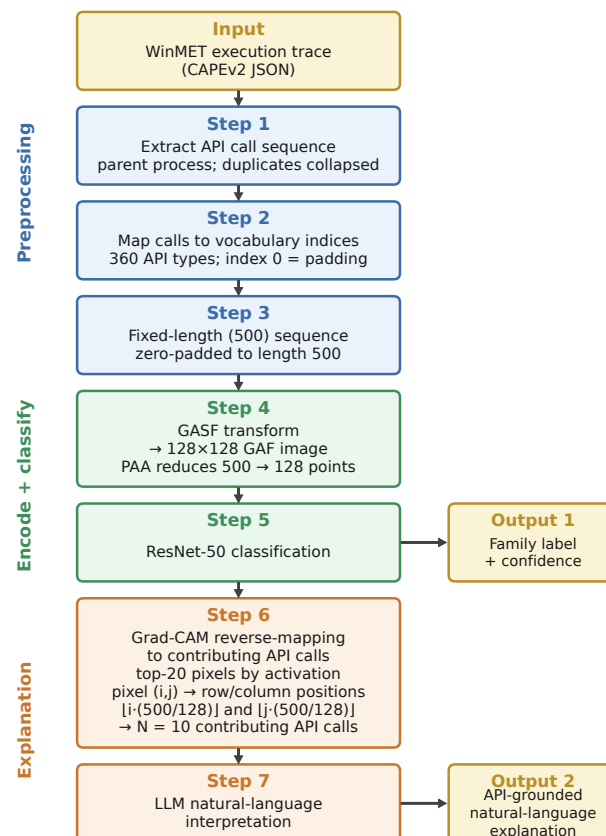


Figure 1. Overall architecture of the proposed explainable classification-and-explanation pipeline (Steps 1–7). Each input yields two coupled outputs: a family label with confidence and an API-grounded natural-language explanation.

Table 3. The seven pipeline steps with their inputs, operations, and outputs.

Step	Input	Operation	Output
1	WinMET JSON	Extract API calls (parent process, all categories; consecutive duplicates collapsed; first 500)	API sequence with per-call fields
2	API sequence	Map call names to integer indices via the vocabulary	Indexed sequence
3	Indexed sequence	Normalize to fixed length with zero-padding	Length-500 sequence
4	Length-500 sequence	Gramian Angular Summation Field transform	128×128 image
5	GAF image	ResNet-50 classification	Family label and confidence
6	Image and model	Grad-CAM; reverse-map activations to API calls	Structured API context
7	API context	LLM natural-language interpretation	API-grounded rationale

3.3. API Sequence Preprocessing (Steps 1–3)

Step 1 (Extraction). From each trace we read `behavior.processes.calls` for the parent process across all categories, retaining the API name, category, arguments, and return value. Consecutive duplicate calls are collapsed (a run of the same API becomes a single entry), and the first 500 calls are kept. Across the ten families used here (16,771 traces; parent process after consecutive duplicate removal and before truncation) the sequence length is strongly right-skewed: the median is 205 calls, with $p_{25} = 175$, $p_{75} = 714$, $p_{90} = 4982$ and a maximum of 74,624. A cap of 500 therefore leaves 54.0% of the traces untouched and truncates the remaining 46.0%, retaining on average 76.7% of each trace. The cap trades retained evidence against explanation granularity: because the GAF image is fixed at 128×128 , one pixel corresponds to 3.91 calls at length 500 against 1.95 at 250 and 7.81 at 1000; Section 4.2 reports an ablation over these three lengths. Seven of the ten families have a mean collapsed length above the cap—from 653 for Vbclone to 27,347 for Cosmu—while Redline (274), Disabler (229) and Berbew (86) fall below it. Child processes are not used by the pipeline; for reference their call counts are far larger (mean of 31,158, median of 4077, maximum of 261,029), so the parent-process convention already sets aside a substantial part of the recorded activity. We keep 500 as this operating point, and follow the parent-process, all-category convention of Zhang et al. [5]. The full per-call information is preserved as `dataset_full_calls.json` for later reverse-mapping.

Step 2 (Vocabulary). Collapsed API call names are collected across all traces and the vocabulary is formed with a frequency cap of 500 types; because only 360 distinct API names occur in WinMET, this cap is not reached and the vocabulary contains 360 types (a vocabulary size of 361 including the padding index). Each name is assigned a unique integer index in `vocab.json`. The index identifies the API type only and carries no ordinal meaning; unknown APIs and padding share index 0. This encoding matches that of Zhang et al. [5].

Step 3 (Fixed-length normalization). Each indexed sequence is normalized to length 500; shorter sequences are zero-padded. This guarantees a common input shape for the GAF transformation.

3.4. GAF Image Transformation (Step 4)

The GAF transformation rescales a one-dimensional series to $[-1, 1]$, represents each value as an angle $\phi = \arccos(\cdot)$ in polar coordinates, and forms the Gramian Angular Sum-

mation Field (GASF), whose entry at position (i, j) is $\cos(\phi_i + \phi_j)$ [6]. Because each image coordinate corresponds to a time position in the original sequence, the encoding expresses *which* API (identified by its unique index) was issued in *what* order (identified by its time position). We compute the GASF with the pyts implementation (method = ‘summation’) at an output size of 128×128 . Each integer-index sequence $\mathbf{x} = (x_1, \dots, x_n)$ (here $n = 500$) is first divided by its per-sample maximum, $x'_t = x_t / \max_s x_s \in [0, 1]$ (Because the pyts GASF routine applies its own min–max rescaling to $[-1, 1]$ (below), this initial division by the per-sample maximum does not change the resulting image—we verified that including or omitting it yields identical GASF images up to numerical precision. We report it here because it is part of the pipeline as run, following the reference implementation of Zhang et al. [5]). For a length- n input, pyts reduces the series to $m = 128$ points by piecewise aggregate approximation (PAA)—the mean of x' over each of m contiguous, equal-width segments—then rescales the reduced series $\tilde{x}$ to $[-1, 1]$ by $\tilde{x}_k = 2(\tilde{x}_k - \tilde{x}_{\min}) / (\tilde{x}_{\max} - \tilde{x}_{\min}) - 1$, encodes each value as an angle $\phi_k = \arccos(\tilde{x}_k)$, and forms the field $\text{GASF}_{ij} = \cos(\phi_i + \phi_j)$. The resulting 128×128 image is a downsampling of about $3.9\times$ per dimension relative to the full 500×500 field, which we adopt for computational efficiency. This order-preserving encoding of API call sequences follows the method of Zhang et al. [5], who first applied GAF to dynamic API call sequences and reported classification-accuracy gains for both MLP and CNN models; we adopt the same encoding and scale it to the larger WinMET dataset. Two caveats are worth noting. First, the vocabulary indices are nominal rather than ordinal, so the magnitude relationships GASF encodes between index values are partly an artifact of how indices are assigned; we therefore treat absolute performance with caution and test sensitivity to the assignment directly (Section 4.2): re-indexing the vocabulary with three random permutations leaves accuracy and AUC-ROC statistically unchanged, with only a minority-class-driven fluctuation in Macro-F1. Second, this does not affect the Experiment 1 contrast, because both encodings use the same index vocabulary and image size, so their comparison isolates the presence or absence of temporal order.

The principal motivation for GAF over a frequency-based encoding is the preservation of call order. The structured heatmap of El Rhayati et al. [7] aggregates token frequencies and therefore discards the order in which API calls occur, whereas GAF retains it. This contrast is the basis of Experiment 1, which isolates the effect of temporal-order preservation on family classification.

Taken together, the use of an image representation in this pipeline rests on three grounds. First, it follows an established and validated line of image-based malware classification, from byte-level malware images [2] to image encodings of dynamic API call sequences [3,5] and structured CAPEv2 heatmaps [7], and it permits the reuse of CNN architectures pretrained on natural images [17] without hand-crafted feature engineering. Second, where image and sequence representations have been compared directly, image encoding has been reported to be competitive rather than inferior: Zhang et al. [5] find that GAF visualization improves classification accuracy over the non-visualized API call sequence for both an MLP and a CNN, so imaging is not a concession on accuracy relative to direct sequence modeling. Third, and decisively for our purpose, a two-dimensional image is the representation over which Grad-CAM [8] produces the spatial saliency that we reverse-map to the contributing API calls; the image encoding is therefore a prerequisite of the explanation method rather than merely an alternative classifier input. Within the family of image encodings, we adopt GAF specifically because, unlike a frequency-only image, it preserves the temporal order of calls [6] that this reverse-mapping requires. We thus justify the image representation on the grounds of precedent, competitiveness, and—above all—explainability, while making no claim that it surpasses sequence models in accuracy—a

comparison we report in Section 4.2, where a 1D-CNN on the raw sequences is statistically comparable to the GAF model on all metrics except a small AUC-ROC difference.

Figure 2 illustrates this contrast on a representative API-index sequence (panel a): the GASF encoding (panel b) is shaped by *where* each call occurs in the sequence, whereas the frequency-only heatmap (panel c) reflects only *how often* each index appears and is therefore unchanged if the calls are reordered.

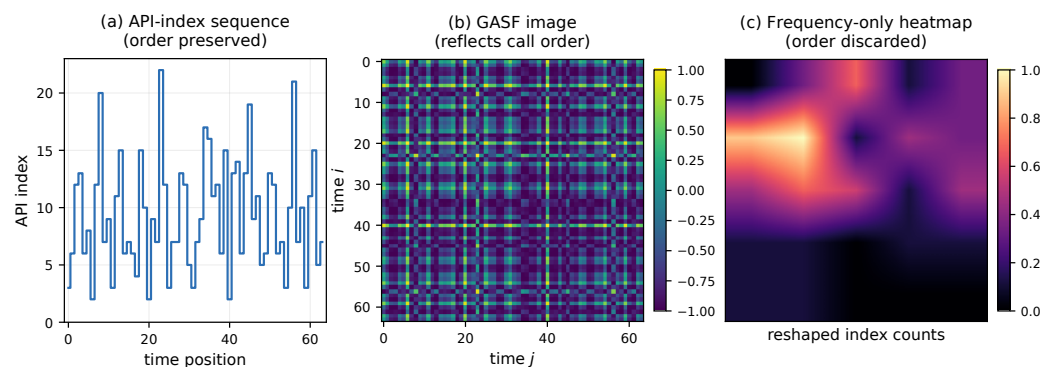


Figure 2. Illustrative comparison of the two encodings on a representative API-index sequence: (a) the sequence, in which call order is preserved; (b) the GASF image, whose structure reflects call order; (c) a frequency-only heatmap, which discards order.

3.5. Classification (Step 5)

We classify the GAF image with a ResNet-50 [17], a 50-layer convolutional network whose residual (skip) connections—with each block learning a residual function added to its input—keep gradients well behaved and allow a network this deep to be trained stably. We initialize from the ImageNet1K-V1 weights and replace the final fully connected layer with a single linear layer to ten outputs (no intermediate dropout); each single-channel 128×128 image is replicated to three channels to match the network input, and no ImageNet mean-variance normalization is applied, so the pixel values remain in $[-1, 1]$. All layers are fine-tuned end-to-end, with no backbone freezing. Training uses the Adam optimizer with default moment parameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, no weight decay) at a fixed learning rate of 1×10^{-4} , a batch size of 32, and 50 epochs, with no learning rate schedule, early stopping, or data augmentation. The loss is class-weighted cross-entropy whose weights are computed from the training split alone, inversely proportional to class frequency (the balanced heuristic), to counter the family imbalance of Table 2. Within each training run we retain the model state with the highest validation accuracy over the 50 epochs.

3.6. Explanation Generation (Steps 6–7)

The explanation stage is the component that distinguishes this pipeline from a classifier and is evaluated as a first-class objective in Section 4.

Step 6 (Grad-CAM Activation and API Reverse-Mapping). For each sample we run Grad-CAM [8] on the predicted class at the final convolutional block (Layer-4) of ResNet-50. This yields a coarse 4×4 activation map, which the Grad-CAM implementation upsamples to the 128×128 image grid by bilinear interpolation. Because the GASF is computed after the PAA reduction from 500 to 128 points described in Section 3.4, each image coordinate i corresponds to the original sequence position $\lfloor i \cdot (500/128) \rfloor$; because a pixel aggregates about 3.91 calls at length 500, this takes the leading call of the corresponding segment as its representative, so the attributed position is unambiguous. We rank the pixels of the upsampled map by activation and take the top 20 (twice the target of $N = 10$) as candidates; each candidate pixel (i, j) contributes two original sequence

positions, $\lfloor i \cdot (500/128) \rfloor$ from its row and $\lfloor j \cdot (500/128) \rfloor$ from its column, which are accumulated in order of decreasing activation, with ties broken by ascending position, until $N = 10$ distinct positions have been obtained; duplicate positions are discarded by this set accumulation. Because each image coordinate indexes a *trace position*—not wall-clock time or a periodic phase—these positions denote where in the order-preserving call sequence the salient calls occur. The mapping granularity depends on the sequence length: each of the 128 pixels corresponds to about 1.95, 3.91, and 7.81 calls at lengths 250, 500, and 1000, so a longer sequence captures more behavior but coarsens the localization of the explanation. For each retained position we retrieve the API name, category, arguments, and return value from `dataset_full_calls.json` to assemble a structured *API context*. Because positions in the collapsed sequence do not coincide with indices in the raw `calls` array, Step 1 stores the full per-call fields alongside the collapsed sequence so that this retrieval is exact. For Experiment 2, we additionally compute a per-family mean faithfulness score from these mappings. Figure 3 illustrates this reverse-mapping.

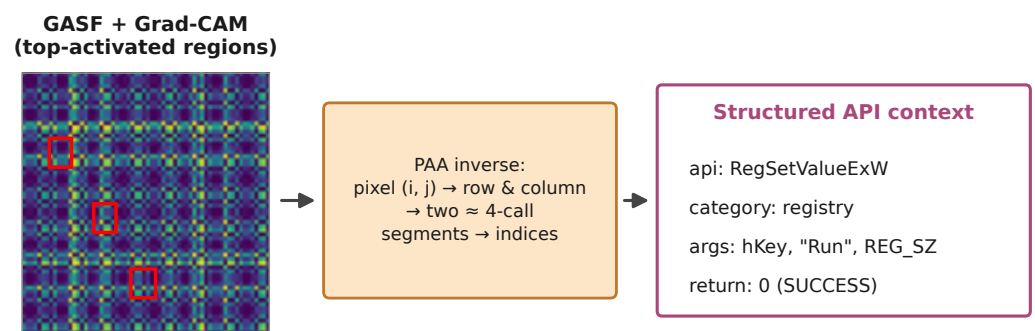


Figure 3. Schematic of the Grad-CAM reverse-mapping (Step 6): activated regions of the GASF image are mapped, through the PAA correspondence, to the contributing API call positions, from which the API name, category, arguments, and return value are retrieved to form the API context. The figure illustrates the process and is not an experimental result.

Step 7 (LLM-Based Natural-Language Interpretation). The API context, together with the predicted family and confidence, is passed to a single LLM (claude-sonnet-4-6) that generates a one- to three-paragraph explanation of why the sample was assigned to the predicted family. The prompt is assembled from a role instruction (a malware-analysis expert); a task instruction that asks the model to explain, in one to three paragraphs, why the sample belongs to the predicted family and how the arguments and return values influenced the decision; the classification result (family and confidence); the API context, listed as the top reverse-mapped calls with their category and—when parameter context is enabled—their arguments and return values; and a domain-knowledge field that is left empty in these experiments so that the compared conditions differ only in prompt regime and parameter context. In the few-shot configuration, a single worked example for the predicted family is appended. Supplying argument- and return-level context is a capability that name-only prior work cannot offer, and Section 4 quantifies its effect on explanation quality. Figure 4 shows the prompt structure.

Table 4 consolidates the configuration of all pipeline stages described above.

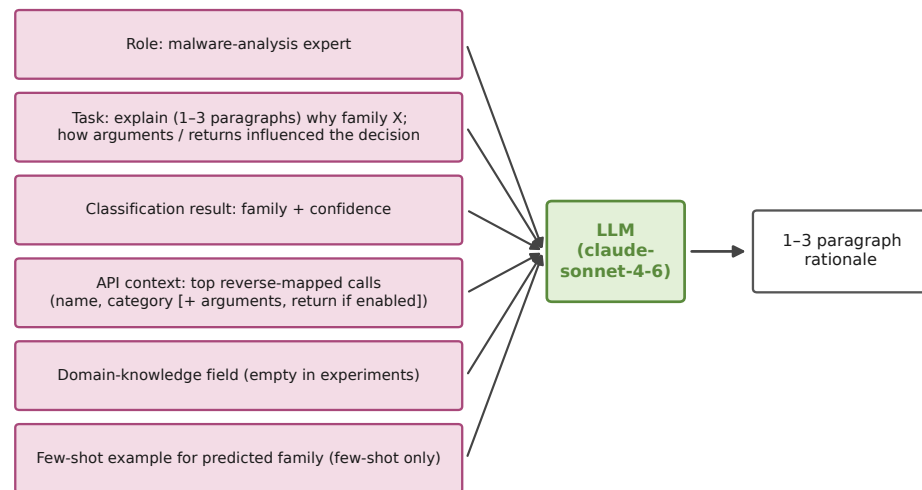


Figure 4. Interpretation prompt (Step 7): A role instruction; a task instruction (explain in one–three paragraphs why the sample is the predicted family and how arguments and return values influenced the decision); the classification result; the API context (with arguments and return values when enabled); a domain-knowledge field (held empty in these experiments); and, in the few-shot configuration, a worked example for the predicted family. The assembled prompt is sent to the LLM, which returns a natural-language rationale.

Table 4. Configuration of the pipeline stages.

Stage	Settings
Preprocessing	Sequence length of 500; consecutive duplicate removal; vocabulary of 360 API types (size of 361 including padding index 0; the top-500 cap is not reached)
GAF	pyts GramianAngularField; method=summation (GASF); image size of 128×128 (PAA reduction from 500)
Classifier	ResNet-50 (ImageNet1K-V1, fine-tuned end-to-end); Adam; learning rate of 1×10^{-4} ; batch size of 32; 50 epochs; class-weighted cross-entropy
Data split	Train/validation/test = 72/8/20; stratified; fixed seed (42)
Grad-CAM	Target layer layer4; top- n = 10 reverse-mapped positions (from the top-2 n activated pixels)
LLM	claude-sonnet-4-6; structured prompt (role, task, classification result, API context, empty domain-knowledge field; optional few-shot example); one–three paragraph output

4. Experiments and Results

4.1. Experimental Setup

Experiments were conducted on a Dell Precision 7920 workstation (Intel Xeon Silver 4110, 96 GB RAM) running Windows 11 with WSL2 and Docker Desktop, ensuring a containerized and reproducible environment. The workstation’s NVIDIA Quadro P620 (2 GB) GPU was not exposed to the container, as GPU passthrough was not configured, so all training and inference were performed on the CPU. The implementation uses PyTorch 2.3.1 and torchvision 0.18.1 from a CUDA 12.1 base image—CUDA-enabled but run in CPU-only mode throughout—together with pyts 0.13.0, grad-cam 1.5.0, the Anthropic SDK ($\geq 0.50.0$) for interpretation, and the OpenAI SDK ($\geq 1.0.0$) for evaluation. The interpretation model is claude-sonnet-4-6 and the judge models are gpt-4o-2024-08-06 and gpt-4.1-2025-04-14, two independent judges run at temperature 0; we report these versions, and the decoding configuration for both is provided with our implementation to support replication despite the stochasticity of LLM generation. Figure 5 shows the

three experiments as a chain: Experiment 1 selects the best classification model, Experiment 2 evaluates its Grad-CAM reverse-mapping, and Experiment 3 evaluates the LLM interpretation built on that API context.

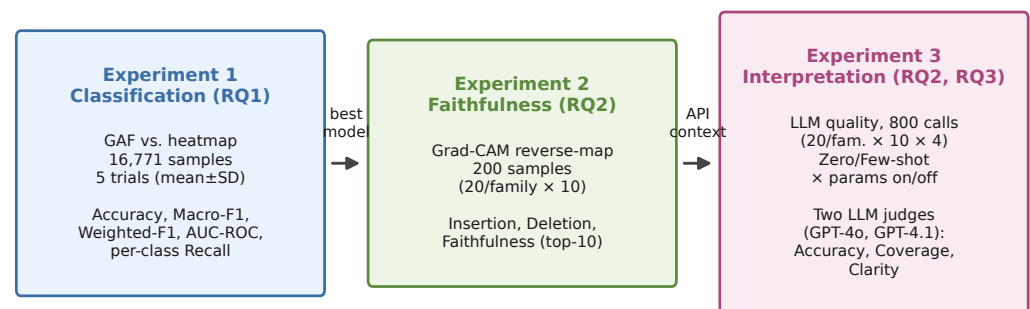


Figure 5. Overall evaluation design: classification, Grad-CAM faithfulness, and LLM interpretation form a chain in which each experiment feeds the next.

We report classification on the held-out test set with accuracy, Macro-F1, Weighted-F1, per-class recall, and One-vs-Rest macro AUC-ROC, each averaged over five trials with standard deviations; differences between the two encodings are assessed with Welch’s two-sample *t*-test over the five trials—appropriate for small, independent samples with possibly unequal variances, and more conservative than the pooled Student’s *t*-test—corroborated by a distribution-free Mann–Whitney *U*-test that assumes no normality. Grad-CAM quality is reported with Insertion, Deletion, and faithfulness (=Insertion – Deletion) scores. Explanation quality is rated for Accuracy, Coverage, and Clarity on a 1–5 scale (15 points maximum).

4.2. Experiment 1: Classification Performance (GAF vs. Heatmap)

Experiment 1 compares two encodings under identical conditions (sequence length 500, image size 128×128 , ResNet-50, WinMET 16,771 samples, five trials). The GAF encoding follows that of Zhang et al. [5] and preserves call order; the heatmap encoding is a simplified frequency-only baseline inspired by El Rhayati et al. [7], not a reproduction of their structured CAPEv2 heatmap method: the occurrences of each of the 361 vocabulary indices (including the padding index 0) are counted over the sequence; the resulting count vector is max-normalized, reshaped into a 19×19 matrix ($19^2 = 361$), and bilinearly resized to 128×128 . This representation retains frequency information only and discards call order. Both encodings produce single-channel 128×128 images that are replicated to three channels for ResNet-50, so that holding the input vocabulary and image size fixed isolates the preservation of temporal order as the factor that differs between them. The five trials are independent training runs on a single stratified train/validation/test split (seed 42); because model initialization and mini-batch ordering are not seed-controlled, the trial-to-trial spread reflects this training stochasticity, and the reported significance tests are computed over these runs. From the five GAF trials, the run with the highest best-validation accuracy (Trial 5, 0.8241) is retained as the final model used in Experiments 2 and 3. Because the two prior methods use different datasets, this is a method comparison on a common dataset rather than a direct comparison of published accuracy values.

Tables 5 and 6 report performance on the held-out test set—the overall metrics and the per-class recall, respectively—for both encodings. The two encodings are statistically indistinguishable on overall accuracy (GAF 0.8021 vs. heatmap 0.8031; Welch’s *t*-test, $p = 0.76$; Mann–Whitney $p = 0.84$), and the heatmap is marginally higher on the One-vs-Rest macro AUC-ROC (0.9647 vs. 0.9620). On this single fixed split, GAF attains a higher Macro-F1 (0.6335 vs. 0.6077, Welch $p = 0.015$, Mann–Whitney $p = 0.016$) and a higher Weighted-F1 (0.7955 vs. 0.7843, Welch $p = 0.012$, Mann–Whitney $p = 0.036$). GAF also attains a higher mean

best-validation accuracy, that is, the peak validation-split accuracy used for model selection (0.8132 ± 0.0059 vs. 0.8103 ± 0.0027). This last quantity is a validation-based criterion measured on a separate split, and we report it as a complementary check on the test-set comparison. Because Macro-F1 weights every family equally, it is most sensitive to the lower-scoring minority families; the 0.0258 Macro-F1 advantage—larger than either encoding’s standard deviation (GAF 0.0114, heatmap 0.0123)—therefore indicates better balanced performance across families under order preservation *on this fixed split*, even though overall accuracy, driven by the majority families, is unchanged. The per-class recall in Table 6 is mixed at the family level: GAF is higher on the hardest family, Noon (0.1207 vs. 0.0264), and on Amadey (0.2967 vs. 0.2366), whereas the heatmap is higher on Disabler and Agenttesla. These per-family values are noisy, however, with large trial-to-trial standard deviations. They reach about 0.15 for GAF, and they are even larger for the heatmap, which varies by about 0.43 on Taskun and 0.37 on Agenttesla. We therefore draw conclusions at the Macro-F1 level rather than from individual families. The low recalls of the minority families Noon, Amadey, and Disabler are a consequence of the pronounced class imbalance (Table 2), which the class-weighted loss only partially offsets. These three families are among the smallest in the dataset: Noon, Amadey, and Disabler contribute 530, 763, and 799 samples, against 4438 for the largest family. Even after reweighting the loss, the model therefore sees comparatively few examples of them, and their per-fold estimates carry higher variance. The same small-sample sensitivity is visible in the index-permutation analysis below, where the only residual fluctuation traced to a single minority family (Amadey); this is why we base our conclusions on Macro-F1 and cross-validated intervals rather than on the recall of any individual minority family.

Table 5. Experiment 1: Classification performance of GAF vs. heatmap encodings on WinMET (mean $\pm$ SD over five trials). The heatmap is a simplified frequency-only baseline inspired by El Rhayati et al. [7], not a reproduction of their structured CAPEv2 method.

Encoding	Accuracy	Macro-F1	Weighted-F1	AUC-ROC	Trials
GAF (ours)	0.8021 ± 0.0043	0.6335 ± 0.0114	0.7955 ± 0.0050	0.9620 ± 0.0028	5
Frequency-only heatmap [7]	0.8031 ± 0.0042	0.6077 ± 0.0123	0.7843 ± 0.0048	0.9647 ± 0.0004	5

Table 6. Experiment 1: Per-class recall for the GAF and heatmap encodings (mean $\pm$ SD over five trials). The best- and worst-performing families by GAF recall (Berbew, Noon) are shown in bold.

Family	GAF Recall	Heatmap Recall
Redline	0.8608 ± 0.0188	0.8536 ± 0.0080
Vbclone	0.9995 ± 0.0006	0.9990 ± 0.0012
Berbew	1.0000 ± 0.0000	1.0000 ± 0.0000
Agenttesla	0.4758 ± 0.0961	0.5250 ± 0.3677
Cosmu	0.9989 ± 0.0022	1.0000 ± 0.0000
Taskun	0.4796 ± 0.0979	0.4889 ± 0.4255
Disabler	0.3588 ± 0.1479	0.4462 ± 0.0829
Amadey	0.2967 ± 0.1503	0.2366 ± 0.0636
Noon	0.1207 ± 0.0649	0.0264 ± 0.0210
Strab	0.7699 ± 0.0251	0.7871 ± 0.0343

To test whether these single-split differences are robust, we add a five-fold stratified cross-validation in which GAF and the heatmap encoding are trained and evaluated on an identical fold partition (StratifiedKFold, $k = 5$, shuffled, seed 42, with per-fold class weights recomputed), so that the two encodings use exactly the same data in every fold. Because each fold then yields paired observations, we apply a paired analysis—the per-fold GAF–heatmap difference with its 95% confidence interval and the paired effect size d_z —rather than the independent-samples Welch’s t -test used for the five independent

trials above; we also compute the Wilcoxon signed-rank test on the per-fold differences (for example, Macro-F1 $p = 0.19$), but with $k = 5$ folds it cannot fall below $p = 0.0625$ and so adds nothing beyond the confidence intervals and d_z , through which we therefore interpret the effect. As Table 7 shows, the two encodings are statistically comparable: the Macro-F1 difference is $+0.009$ with a 95% CI of $[-0.005, +0.023]$ ($d_z = 0.79$), while accuracy and best-validation accuracy are likewise indistinguishable (their intervals include zero). The only intervals excluding zero are Weighted-F1 (GAF higher by 0.004) and AUC-ROC (heatmap higher by 0.005), both negligibly small and in opposite directions. The single-split Macro-F1 advantage therefore does not persist under resampling, and we characterize the two encodings as comparable in classification performance; GAF is retained not for its performance advantage but for its order-preserving property, which the Grad-CAM reverse-mapping in Step 6 requires. Because seeds are not fixed, this cross-validation also varies model initialization and mini-batch ordering across folds, so the comparison holds across realistic training stochasticity as well as across data splits.

Table 7. Experiment 1: Five-fold stratified cross-validation of GAF vs. heatmap on WinMET (mean $\pm$ SD over five folds). The right-hand columns give the paired per-fold GAF–heatmap difference with its 95% CI and the paired effect size d_z ; a positive value favors GAF.

Metric	GAF	Heatmap	Diff [95% CI]	d_z
Accuracy	0.7978 $\pm$ 0.0068	0.8039 $\pm$ 0.0030	−0.006 [−0.017, +0.005]	−0.67
Macro-F1	0.6196 $\pm$ 0.0182	0.6108 $\pm$ 0.0182	+0.009 [−0.005, +0.023]	+0.79
Weighted-F1	0.7872 $\pm$ 0.0066	0.7837 $\pm$ 0.0074	+0.004 [+0.001, +0.007]	+1.52
AUC-ROC	0.9596 $\pm$ 0.0033	0.9648 $\pm$ 0.0006	−0.005 [−0.010, −0.001]	−1.41
Best-Val-Acc	0.8066 $\pm$ 0.0087	0.8043 $\pm$ 0.0077	+0.002 [−0.004, +0.008]	+0.47

As a direct sequence-model baseline, we additionally train a 1D-CNN on the same integer-index sequences under the settings used for ResNet-50 (Adam, learning rate 10^{-4} , batch 32, 50 epochs, class-weighted cross-entropy, five trials on the same split). The model embeds each API index (vocab = 361, padding_idx = 0) and applies three stacked 1D convolutions with global max-pooling and a linear head for 3,113,738 parameters—about one-eighth of the ResNet-50 classifier (approximately 23.5 million). Table 8 compares the two over five trials. The 1D-CNN is marginally higher on every metric, but the differences are small (0.003–0.007) and statistically significant on only one axis, AUC-ROC (Welch $p = 0.022$, Mann–Whitney $p = 0.047$, 95% CI of the difference [+0.001, +0.010]); accuracy, Macro-F1, and best-validation accuracy are statistically indistinguishable under both tests, while Weighted-F1 is the closest to being significant under Welch’s test ($p = 0.0515$)—still above the 0.05 threshold—and clearly non-significant under the rank-based Mann–Whitney test ($p = 0.15$). That a sequence model matches the image classifier with far fewer parameters is expected and consistent with our stated scope: we do not claim an accuracy advantage for image encoding. The image representation is instead a prerequisite of the explanation method—Grad-CAM produces spatial saliency over the GAF image that we reverse-map to the contributing API calls, whereas the 1D-CNN collapses the sequence with global pooling and therefore cannot localize the responsible calls. This comparison also serves as an empirical test of the premise, cited from Zhang et al. [5], that GAF imaging need not cost accuracy: on the larger WinMET dataset the two are comparable, with the sequence model marginally ahead only on AUC-ROC.

Table 8. Experiment 1: GAF+ResNet-50 vs. a 1D-CNN sequence-model baseline on WinMET (mean $\pm$ SD over five trials). Differences (1D-CNN–GAF) are assessed with Welch’s *t*-test and a distribution-free Mann–Whitney *U* test; only AUC-ROC differs significantly.

Metric	GAF+ResNet-50	1D-CNN	Welch <i>p</i>	MW <i>p</i>
Accuracy	0.8021 $\pm$ 0.0043	0.8068 $\pm$ 0.0062	0.26	0.31
Macro-F1	0.6335 $\pm$ 0.0114	0.6369 $\pm$ 0.0087	0.65	0.84
Weighted-F1	0.7955 $\pm$ 0.0050	0.8028 $\pm$ 0.0039	0.0515	0.15
AUC-ROC	0.9620 $\pm$ 0.0028	0.9675 $\pm$ 0.0026	0.022	0.047
Best-Val-Acc	0.8132 $\pm$ 0.0059	0.8201 $\pm$ 0.0028	0.08	0.12

Sequence-length ablation. To test how sensitive the pipeline is to the fixed length of 500 chosen in Step 1, we repeat GAF training at lengths 250 and 1000, with five trials each, with every other setting unchanged; the length-500 group reuses the five GAF runs of Table 5 rather than retraining. Table 9 reports the result. A Kruskal–Wallis test across the three lengths is significant for accuracy, Macro-F1, Weighted-F1 and best-validation accuracy, but not for AUC-ROC. Post hoc two-sided Mann–Whitney tests localize the effect at length 1000; these tests are exact when the compared samples contain no ties, and they fall back on a tie-corrected normal approximation otherwise. Against 500, length 1000 is higher on accuracy (+0.012, $p = 0.012$), Macro-F1 (+0.035, $p = 0.008$), Weighted-F1 (+0.014, $p = 0.032$) and best-validation accuracy (+0.013, $p = 0.032$). Lengths 250 and 500, by contrast, differ on no metric (smallest $p = 0.151$), and AUC-ROC runs the other way, being numerically lowest at 1000. A length of 500 is therefore *not* the performance optimum, which is consistent with 46.0% of the traces being truncated at that cap. The gain at 1000 is unsurprising given how right-skewed the length distribution is: the median trace has 205 calls, but the upper quartile reaches 714 and the ninetieth percentile reaches 4982, so raising the cap recovers evidence from precisely the long traces that 500 curtails.

Table 9. Experiment 1: Sequence-length ablation on WinMET (GAF encoding; mean $\pm$ SD over five trials per length; the length-500 column reuses the runs of Table 5). The last column reports a Kruskal–Wallis test across the three lengths.

Metric	Length 250	Length 500	Length 1000	Kruskal–Wallis
Accuracy	0.7990 $\pm$ 0.0067	0.8021 $\pm$ 0.0043	0.8138 $\pm$ 0.0029	$H = 9.59$, $p = 0.008$
Macro-F1	0.6219 $\pm$ 0.0033	0.6335 $\pm$ 0.0114	0.6687 $\pm$ 0.0099	$H = 10.50$, $p = 0.005$
Weighted-F1	0.7882 $\pm$ 0.0054	0.7955 $\pm$ 0.0050	0.8095 $\pm$ 0.0056	$H = 9.38$, $p = 0.009$
AUC-ROC	0.9647 $\pm$ 0.0011	0.9620 $\pm$ 0.0028	0.9594 $\pm$ 0.0042	$H = 4.34$, $p = 0.114$
Best-Val-Acc	0.8092 $\pm$ 0.0062	0.8132 $\pm$ 0.0059	0.8266 $\pm$ 0.0043	$H = 8.80$, $p = 0.012$

We nevertheless retain 500, because the quantity this study is built to deliver is not accuracy but attribution: one pixel of the GAF image covers 3.91 calls at length 500 and 7.81 at 1000, so doubling the sequence halves the resolution at which Grad-CAM activations can be tied to individual API calls. Shortening to 250 sharpens the mapping to 1.95 calls per pixel, but the point estimates are lower than at 500 on four of the five metrics, so a shorter sequence is not free either. The choice of 500 therefore balances two quantities that move in opposite directions: the fraction of each trace that survives truncation, and the number of calls that a single pixel represents. At this cap 54.0% of the traces are retained in full and 76.7% of each trace is retained on average, while every pixel still maps to fewer than four calls, so the ten positions that Step 6 reports remain individually identifiable. With

five runs per group these tests are coarse—the smallest p the chi-square approximation can return here is 0.0019—so they establish a consistent ordering rather than a precisely estimated effect.

Sensitivity to the index assignment. A reviewer noted that the vocabulary indices are nominal, yet the GAF encoding treats them as ordered magnitudes through the frequency-based assignment of Step 1. To test whether the pipeline depends on this particular assignment, we re-index the vocabulary with three independent random permutations (the padding symbol is held fixed at 0), re-encode every trace, and retrain the GAF model five times per permutation, leaving all other settings unchanged; the current (frequency-ordered) column reuses the five GAF runs of Table 5. Table 10 reports the outcome. Classification performance is essentially unchanged: a Kruskal–Wallis test across the four assignments is non-significant for accuracy ($p = 0.72$), AUC-ROC ($p = 0.18$) and best-validation accuracy ($p = 0.52$), and pooling the three permutations ($n = 15$) leaves every metric within 0.006 of the frequency-ordered baseline (all $p > 0.05$). The Kruskal–Wallis test is marginally significant for Macro-F1 ($p = 0.044$) and Weighted-F1 ($p = 0.022$), but this is driven entirely by permutation 1: removing it, the remaining three assignments are statistically indistinguishable (Macro-F1 $p = 0.76$, Weighted-F1 $p = 0.69$), and only permutation 1 differs from the baseline on any metric (Weighted-F1, $p = 0.032$). The effect traces to the trial-to-trial variability in a minority family (Amadey), whose mean recall happens to be low under permutation 1 (0.18 against 0.41 and 0.38 under permutations 2 and 3), rather than to any systematic degradation—permutations 2 and 3 match or slightly exceed the baseline. The robustness is therefore clearest on accuracy and AUC-ROC, the metrics least sensitive to minority-class fluctuation. This indicates that GAF+ResNet-50 captures structural patterns in the call sequence rather than depending on the specific, arbitrary frequency-based index layout.

Table 10. Experiment 1: Sensitivity to the API-to-index assignment on WinMET (GAF encoding; mean $\pm$ SD over five trials per assignment; the frequency-ordered column reuses the runs of Table 5). The last column reports a Kruskal–Wallis test across the four assignments.

Metric	Freq.-Ordered	Perm. 1	Perm. 2	Perm. 3	Kruskal–Wallis
Accuracy	0.8021 $\pm$ 0.0043	0.7951 $\pm$ 0.0105	0.8028 $\pm$ 0.0036	0.7992 $\pm$ 0.0057	$p = 0.72$
Macro-F1	0.6335 $\pm$ 0.0114	0.6144 $\pm$ 0.0141	0.6365 $\pm$ 0.0055	0.6375 $\pm$ 0.0030	$p = 0.044$
Weighted-F1	0.7955 $\pm$ 0.0050	0.7836 $\pm$ 0.0059	0.7977 $\pm$ 0.0023	0.7957 $\pm$ 0.0026	$p = 0.022$
AUC-ROC	0.9620 $\pm$ 0.0028	0.9596 $\pm$ 0.0009	0.9580 $\pm$ 0.0015	0.9580 $\pm$ 0.0049	$p = 0.18$
Best-Val-Acc	0.8132 $\pm$ 0.0059	0.8066 $\pm$ 0.0078	0.8069 $\pm$ 0.0041	0.8091 $\pm$ 0.0071	$p = 0.52$

4.3. Explanation Evaluation

This subsection evaluates the explanation-generation stage of the pipeline directly, treating it as a primary objective rather than an auxiliary visualization.

4.3.1. Experiment 2: Grad-CAM Faithfulness Across Families

Using the best Experiment 1 model, we evaluate the faithfulness of the Grad-CAM saliency that underlies the reverse-mapping on 20 samples per family across the ten families (200 samples). Insertion and Deletion are computed directly on the 128×128 GAF image, masking its pixels in order of Grad-CAM activation—computed for the predicted class, as in Step 6—over ten equal steps (the top 10%, 20%, ..., 100% of pixels by activation) and recording the softmax confidence of the true label at each step; each score is the mean confidence over the ten steps, with no re-encoding from the API sequence. For Insertion we start from a blank (all-zero) image and progressively restore the most-activated pixels, so a faithful saliency map keeps the confidence high; for Deletion we start from the full image and progressively zero the most-activated pixels, so a faithful map should lose

confidence and a low Deletion score is therefore desirable. Faithfulness is their difference (Insertion minus Deletion), so that higher values indicate more faithful saliency. The resulting per-family scores are averaged over the 20 samples.

Table 11 reports the per-family Insertion, Deletion, and faithfulness scores (faithfulness is computed as Insertion – Deletion from high-precision values, so a row’s displayed faithfulness may differ from the difference in its rounded Insertion and Deletion by up to 0.0001). Faithfulness varies markedly across families, from 0.0755 for Noon to 0.7981 for Berbew, with a mean of 0.2469 (mean Insertion 0.5487, mean Deletion 0.3018). Two near-perfectly classified families behave differently: Berbew, which ResNet-50 classifies perfectly (Table 6), also yields the most faithful saliency (0.7981), indicating sharply localized decisive regions, whereas Cosmu, also classified almost perfectly, combines high Insertion (0.8986) with high Deletion (0.7379) and therefore low Faithfulness (0.1607)—accurate classification alone does not guarantee localized saliency. The lowest faithfulness, Noon (0.0755), coincides with the lowest classification recall, consistent with a diffuse behavioral signature.

Table 11. Experiment 2: Per-family Grad-CAM Insertion, Deletion, and faithfulness scores (20 samples/family, 200 total). The families with the highest and lowest faithfulness (Berbew, Noon) are shown in bold.

Family	Redline	Vbclone	Berbew	Agenttesla	Cosmu
Insertion	0.4367	0.9390	0.8877	0.3666	0.8986
Deletion	0.2599	0.5897	0.0896	0.1887	0.7379
Faithfulness	0.1768	0.3493	0.7981	0.1779	0.1607
Family	Taskun	Disabler	Amadey	Noon	Strab
Insertion	0.3246	0.2339	0.5353	0.2346	0.6298
Deletion	0.1370	0.1323	0.2971	0.1590	0.4268
Faithfulness	0.1876	0.1016	0.2382	0.0755	0.2030

Faithfulness baselines. To test whether the Grad-CAM faithfulness above is better than chance and how it compares to an established attribution method, we add three inference-only baselines on the same 200 samples; Table 12 reports the results. A *random-saliency* baseline replaces the Grad-CAM map with uniform random maps (fixed per-sample seeds); an *occlusion* baseline scores each 16×16 patch by the predicted-class confidence drop when it is masked; and a *sequence-level perturbation* test addresses the reviewer’s off-manifold concern directly by masking the reverse-mapped API positions in the sequence (PAD-replacement with the length preserved) and re-encoding, comparing the predicted-class confidence drop for the Grad-CAM-selected positions against an equal-size random control drawn from the real (non-padding) calls. Grad-CAM is substantially more faithful than random saliency (mean faithfulness 0.2469 vs. 0.0537; Wilcoxon signed-rank $p < 10^{-28}$; Grad-CAM higher on 181/200 samples), confirming that the reverse-mapping rests on non-random saliency. Against occlusion, the two are statistically indistinguishable (0.2469 vs. 0.2492; $p = 0.65$; Grad-CAM higher on 95/200): Grad-CAM does not outperform this established method, but matching it indicates that the Grad-CAM saliency is of comparable faithfulness at a fraction of the cost (one backward pass versus 64 masked forward passes per sample). In the sequence-level test, masking the Grad-CAM-selected positions and masking equally as many random real positions produces statistically indistinguishable confidence drops (mean 0.0326 vs. 0.0419; $p = 0.72$; Grad-CAM larger on 99/200); the median drop is near zero for both (≈ 0.001), showing that removing ten calls while preserving sequence length rarely changes the prediction, so this test does not provide additional evidence for reverse-mapping under the length-preserving perturbation. We report these outcomes without adjustment: the reverse-mapping is well supported against a random baseline and on par with occlusion, while the length-preserving sequence

perturbation is inconclusive. This last null result is expected rather than evidence against the reverse-mapping: masking the ten reverse-mapped positions perturbs only about 2% of a length-500 sequence (10 of 500), and the piecewise aggregate approximation that forms each pixel averages roughly 3.91 calls, so the influence of any ten individual positions is diluted before it reaches the classifier—consistent with the near-zero median confidence drop for both the Grad-CAM and the random positions. The faithfulness evidence for the reverse-mapping therefore rests on the image space, where Grad-CAM is decisively more faithful than random saliency ($p < 10^{-28}$), and is not on a length-preserving sequence perturbation that is too small and too averaged to discriminate.

Table 12. Experiment 2: Grad-CAM faithfulness baselines on the 200 samples. Faithfulness is Insertion minus Deletion (higher is more faithful); the sequence-perturbation columns report the predicted-class confidence drop when the reverse-mapped positions (Grad-CAM) or equally as many random real positions are masked. p -values are Wilcoxon signed-rank (Grad-CAM vs. each baseline).

Comparison	Grad-CAM	Baseline	Wilcoxon p
Faithfulness vs. random saliency	0.2469	0.0537	$<10^{-28}$
Faithfulness vs. occlusion	0.2469	0.2492	0.65
Seq. confidence drop vs. random positions	0.0326	0.0419	0.72

4.3.2. Experiment 3: LLM Interpretation Quality

Experiment 3 evaluates the natural-language interpretation under four conditions formed by crossing prompt regime (zero-shot vs. few-shot, in which a single worked example for the predicted family is supplied) with API parameter context (with vs. without arguments and return values; when absent, only the API name and category are shown) for 20 samples per family across ten families and four conditions (800 LLM calls). The domain-knowledge field is held empty throughout, so the conditions differ only in prompt regime and parameter context. To avoid a circular evaluation, the generating model (Claude) and the judging model differ: we adopt an LLM-as-a-Judge protocol [15] in which two independent judges (GPT-4o and GPT-4.1) each score every explanation for Accuracy, Coverage, and Clarity on a 1–5 scale. Here, Accuracy asks whether the explanation correctly justifies the predicted family from the listed API calls, Coverage whether it addresses the key reverse-mapped calls and their significance, and Clarity whether it is clearly written and easy to follow. Because generation and evaluation use different LLMs and no reference text is required, this protocol provides an objective, reference-free measure of quality. We deliberately avoid BLEU/ROUGE, which require reference texts and overlook semantics.

Table 13 reports the scores by condition for both judges (GPT-4o and GPT-4.1) (judge scores are integers (1–5 per criterion)); the condition- and family-level values reported here and in Table 14 are arithmetic means over the 800 evaluations produced by each judge (per condition, with 200 per condition per judge), so a row's listed criteria may sum to its listed total only up to rounding). Supplying API arguments and return values did not improve explanation quality under either judge; on the contrary, the without-parameter conditions scored significantly higher overall (with- vs. without-parameter totals: GPT-4o 12.37 vs. 13.22, two-sided Mann–Whitney $p = 1.8 \times 10^{-16}$, Cohen's $d = 0.58$; GPT-4.1 12.88 vs. 13.96, $p = 1.3 \times 10^{-13}$, $d = 0.61$), with the condition ordering identical for both judges. The criterion-level breakdown in Table 13 shows that the penalty is spread across all three criteria rather than confined to one, and that its locus differs by judge. Under GPT-4o the largest decline is on Clarity (4.76 $\rightarrow$ 4.37 zero-shot and 4.65 $\rightarrow$ 4.34 few-shot), whereas under GPT-4.1 it is on Coverage (4.54 $\rightarrow$ 4.04 and 4.46 $\rightarrow$ 3.96), with Clarity being the least affected criterion for that judge (4.96 $\rightarrow$ 4.80 and 4.95 $\rightarrow$ 4.72). Accuracy declines as well under both judges (4.45 $\rightarrow$ 4.17 and 4.37 $\rightarrow$ 4.19 for GPT-4o; 4.57 $\rightarrow$ 4.14 and

4.45 $\rightarrow$ 4.09 for GPT-4.1), so the additional raw argument text is judged not merely as more verbose but also as less accurate. The few-shot regime did not outperform zero-shot either, so the best condition overall was zero-shot without parameters (13.34/15 for GPT-4o). Interpretation quality also varied by family: Table 14 reports the per-family means over the four conditions, with the GPT-4o total ranging from 12.01/15 for Taskun to 13.51/15 for Vbclone; Berbew, previously the lowest at 8.80, recovers to 13.20 once the corrected API context is supplied—a spread we revisit in Section 5.7.

Table 13. Experiment 3: LLM interpretation quality by condition, scored by two independent judges (GPT-4o and GPT-4.1), 1–5 per criterion, using the corrected API context. For each judge, the highest total across the four conditions is shown in bold.

Prompt	Params	GPT-4o				GPT-4.1			
		Acc	Cov	Cla	Tot	Acc	Cov	Cla	Tot
Zero-shot	without	4.45	4.12	4.76	13.34	4.57	4.54	4.96	14.07
Zero-shot	with	4.17	3.87	4.37	12.40	4.14	4.04	4.80	12.98
Few-shot	without	4.37	4.09	4.65	13.11	4.45	4.46	4.95	13.85
Few-shot	with	4.19	3.81	4.34	12.35	4.09	3.96	4.72	12.79

Table 14. Experiment 3: Per-family LLM interpretation quality (GPT-4o criteria and total; GPT-4.1 total for comparison) averaged over the four conditions. The best- and worst-performing families by GPT-4o total (Vbclone, Taskun) are shown in bold.

Family	GPT-4o			GPT-4.1	
	Acc	Cov	Cla	Total/15	Total/15
Redline	4.49	4.14	4.61	13.24	13.91
Vbclone	4.51	4.33	4.67	13.51	13.88
Berbew	4.42	4.08	4.70	13.20	13.34
Agenttesla	4.08	3.80	4.31	12.19	13.16
Cosmu	4.22	3.51	4.40	12.14	12.47
Taskun	4.01	3.69	4.31	12.01	12.69
Disabler	4.39	4.09	4.65	13.12	14.43
Amadey	4.41	4.15	4.69	13.25	13.81
Noon	4.03	3.84	4.31	12.18	13.16
Strab	4.36	4.11	4.66	13.14	13.38

4.3.3. Human Expert Evaluation of Explanations

The automated judge can reward a fluent but over-attributed rationale. Also, only the report together with the API names and categories is shown, and not the classifier’s confidence, nor the arguments and return values that the interpreter received. We therefore complement the judge-based scores with a human expert evaluation. The authors, as malware-analysis experts, scored 40 generated reports on the same 1–5 Accuracy/Coverage/Clarity scale, blind to the judge scores. The 40 reports comprise ten families $\times$ the four Experiment 3 conditions, and they include the Vbclone case study sample of Section 5.5. Scoring follows a pre-specified rubric in which a rationale is penalized on accuracy when it presents a generic Windows API as family-specific behavior.

The evaluation confirms that over-attribution is pervasive. The mean expert accuracy is only 1.75/5 (mean total 7.97/15), and just two of the 40 reports are judged as well grounded (accuracy ≥ 3). Both come from Berbew under the two with-argument conditions, where the rationale correctly ties InProcServer32 and a portable-executable read to COM-based persistence.

Two properties of the pipeline are nonetheless visible in the expert scores. First, the rationales track the classifier’s confidence. Reports generated for low-confidence predictions

(<0.5) are more accurate (1.94) than those generated for high-confidence predictions (≥ 0.9 ; 1.55), a difference that is significant under a one-sided Mann–Whitney test ($p = 0.010$; two-sided $p = 0.020$). The reason is that low-confidence reports tend to hedge and to acknowledge the absence of family-specific behavior rather than to assert it. We count a report as acknowledging uncertainty when it explicitly qualifies the confidence or the evidence, notes the absence or genericness of family-specific behavior, or hedges with *may/might*. Under this definition, 15 of the 16 low-confidence reports do so (94%) against 2 of the 20 high-confidence reports (10%). This difference is significant by Fisher’s exact test ($p < 10^{-6}$). Second, the expert scores point in the direction of better grounding when arguments are supplied, although the evidence here is weaker. Expert accuracy is higher with arguments than without (1.90 vs. 1.60), but the comparison reaches significance only under a one-sided test ($p = 0.048$; two-sided $p = 0.096$). We therefore read this as directional evidence rather than as an established effect. This does not contradict the judge-based result of Experiment 3. The judges, which are not shown the arguments, score the with-argument reports lower on every criterion (Table 13). Taken together, the two observations suggest that the arguments may carry grounding value the judges cannot see.

Comparing the two evaluation modes exposes the limitation of the automated judge. The judges agree with the experts on Clarity (expert 4.00 vs. GPT-4o 4.53 and GPT-4.1 4.83): the reports are indeed readable. On accuracy they do not. Neither judge’s accuracy distinguishes low- from high-confidence reports (two-sided Mann–Whitney: GPT-4o $p = 0.157$ and GPT-4.1 $p = 0.188$), and neither distinguishes with- from without-argument reports (GPT-4o $p = 0.124$). These three comparisons remain non-significant under a one-sided test as well ($p = 0.078, 0.094$, and 0.062). Judge accuracy is, moreover, essentially uncorrelated with expert accuracy across the 40 reports: Spearman $\rho = -0.08$, $p = 0.62$ for GPT-4o, and $\rho = 0.15$ and $p = 0.34$ for GPT-4.1. The Vbclone case is the sharpest illustration. Both judges award it a near-perfect total (15/15 in the without-argument conditions), whereas the experts score its accuracy 1 in all four conditions (Coverage 2). The reason is that the cited calls—`GetModuleHandleA`, `LdrLoadDll`, and the native memory-management calls—are generic to most Windows programs. Figure 6 summarizes these patterns.

The divergence is structural rather than a failure of the judge: assessing whether a rationale is faithful requires confidence and the arguments, and the judge is given neither. The judge is therefore a reasonable proxy for readability but not for faithfulness, so expert evaluation—or a judge with access to the same evidence—is needed to detect over-attribution. This evaluation has clear limitations. It was performed by the authors as domain experts, was small-scale ($n = 40$), and did not measure inter-rater reliability. Scoring was, however, blind to the judge scores and followed a pre-specified rubric, and two features of the design worked against a self-serving bias. First, the authors’ mean expert total (7.97/15) is well below both automated judges on the same 40 reports: 12.6 for GPT-4o and 12.9 for GPT-4.1. These are the judge means exactly over the reports scored by the human expert, and they therefore differ from the by-condition averages in Table 13. The human scores are thus markedly harsher on the pipeline than the judges rather than being more lenient. Second, the accuracy penalty follows an objective, pre-specified rule: generic Windows APIs presented as family-specific behavior are penalized. A larger evaluation with multiple independent expert annotators is left for future work.

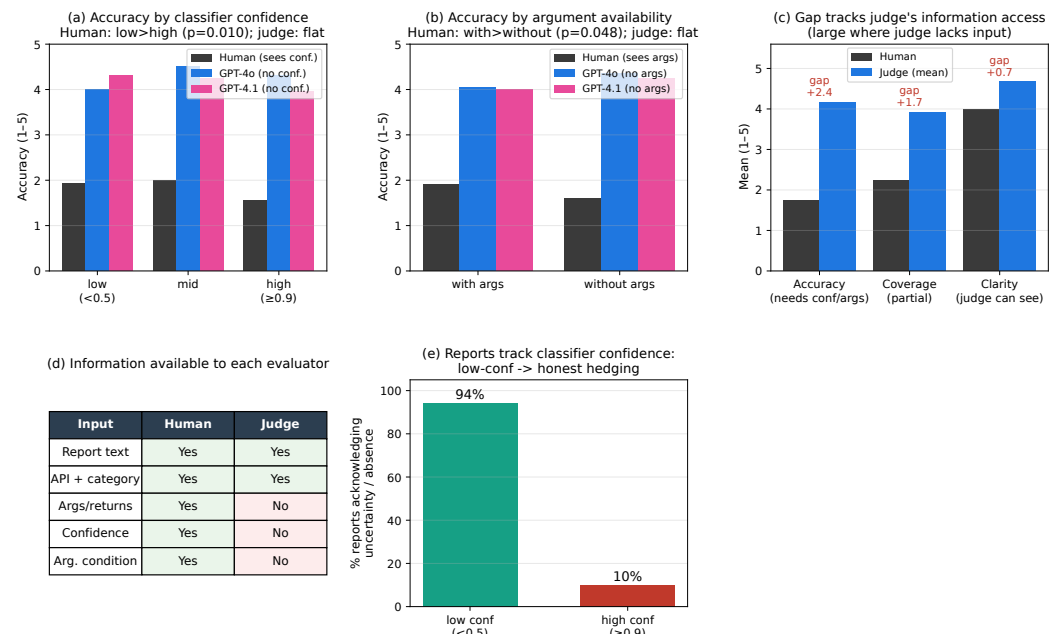


Figure 6. Human expert versus automated-judge scores over 40 reports. The judge receives neither the classifier confidence nor the arguments (an input asymmetry); it agrees with the experts on Clarity but cannot discriminate faithfulness across the confidence and argument axes. The p -values shown in panels (a,b) are one-sided Mann–Whitney values; for two-sided values, the confidence comparison remains significant ($p = 0.020$) whereas the argument comparison does not ($p = 0.096$), so panel (b) should be read as directional evidence rather than an established effect (Section 4.3.3).

5. Discussion

Our experiments answer the first three research questions and yield three headline findings. First, preserving call order with the GAF encoding yields a Macro-F1 edge over a frequency-only heatmap on a single split that does not survive cross-validation, so the two encodings are comparable in classification while overall accuracy is essentially unchanged (RQ1). Second, the faithfulness of the Grad-CAM reverse-mapping varies widely across families, so the API-grounded explanation is dependable for some families and must be read cautiously for others (RQ2). Third, a single LLM agent produces interpretations that automated judges score highly (best condition 13.34/15 under GPT-4o), yet supplying raw argument-level context significantly *reduces* that score under two independent judges (RQ2, RQ3). We discuss each finding in turn, examine how they align at the family level, present a qualitative case study, and close with the implications and limitations.

5.1. Effect of Temporal-Order Preservation (RQ1)

Experiment 1 isolates a single factor: whether preserving the order of API calls benefits family classification. Because GAF retains order while the heatmap retains only frequency, a gap in Macro-F1 in favor of GAF would provide direct evidence that order carries discriminative signal for WinMET families. In our runs this is only partially borne out: on the single fixed split GAF improves Macro-F1 by 0.026 (0.6335 vs. 0.6077; Welch's t -test, $p = 0.015$), but this advantage does not persist under five-fold cross-validation (Table 7; paired difference +0.009, 95% CI $[-0.005, +0.023]$, $d_z = 0.79$), so the two encodings are best described as comparable in classification performance, while overall accuracy is statistically tied (0.8021 vs. 0.8031, $p = 0.76$) and AUC-ROC is marginally higher for the heatmap (0.9620 vs. 0.9647). The single-split Macro-F1 gap is suggestive of a balanced, per-family benefit from order preservation—Macro-F1 being the more meaningful axis under the heavy WinMET class imbalance reported in Section 3.1, since accuracy can

stay high while the rarer families are misclassified and only a metric that weights every family equally exposes that gap—but, as the cross-validation shows, it is not robust enough to establish a classification advantage. The easily separated majority families (Vbclone, Berbew, and Cosmu reach near-perfect recall under both encodings) keep overall accuracy high regardless of encoding. Because the two encodings are thus complementary rather than one strictly dominating the other, the order-preserving property remains valuable beyond classification: the reverse-mapping in Step 6 depends on a meaningful time axis, which only GAF provides.

It is worth situating the absolute accuracy of about 0.80 in context. Image-based malware-family classifiers report substantially higher figures on other corpora—for example, El Rhayati et al. [7] reach 98.4% accuracy with a ResNeXt-50 on the ten-family Avast-CTU CAPEv2 dataset, and Zhang et al. [5] report that GAF visualization improves CNN accuracy for binary detection—but such numbers are not directly comparable to ours, because they are obtained on different datasets with different family sets, class balances, and sandbox configurations, and El Rhayati et al. further caution that random splits tend to overestimate accuracy relative to chronological ones. WinMET is a recently released dataset whose contribution is execution traces and their consensus labels [11]; to the best of our knowledge it does not yet have an established image-based family-classification baseline. We therefore report a controlled within-dataset comparison rather than a cross-dataset accuracy claim, consistent with our objective of demonstrating and characterizing the pipeline rather than maximizing or benchmarking accuracy. The heavy class imbalance of our ten-family subset (about 9.5:1 between the largest and smallest family) further makes raw accuracy an incomplete summary and is the reason we foreground the per-family (Macro-F1) view.

5.2. *Why Faithfulness Varies Across Families (RQ2)*

Experiment 2 quantifies, per family, how faithfully the Grad-CAM reverse-mapping reflects the classifier's decision. A plausible expectation is that faithfulness is higher for families whose behavior concentrates in a few characteristic API calls and is lower for families whose behavior is diffuse; the data partially support this. Berbew attains the highest faithfulness (0.7981) and is classified perfectly (Table 6), indicating that its decisive regions are sharply localized in the GAF image; Noon attains the lowest faithfulness (0.0755) together with the lowest classification recall, consistent with a diffuse signature that the saliency cannot localize. Faithfulness is not, however, a simple function of accuracy: Cosmu is classified almost perfectly yet has low faithfulness (0.1607), because zeroing its most-activated pixels barely lowers the prediction (Deletion 0.7379)—its evidence is redundant rather than concentrated. The wide spread (mean 0.2469, range 0.0755–0.7981) implies that the reverse-mapping is dependable for some families but should be read cautiously for others. Raising and stabilizing this faithfulness—and, where it remains low, flagging the affected families rather than trusting them by default—is a concrete target that we take up in future work (Section 5.7).

5.3. *Value of Argument-Level Context for Interpretation (RQ2, RQ3)*

Experiment 3 first establishes that a single LLM agent does produce interpretations from the Grad-CAM-derived API context: across the four conditions the judges score them between 12.35 and 13.34 out of 15 (GPT-4o) and between 12.79 and 14.07 (GPT-4.1), with Clarity being the strongest criterion and Coverage the weakest. This answers RQ3 in the affirmative, while how far such scores reflect genuine correctness—rather than mere fluency—is qualified by the case study (Section 5.5). The more pertinent question is whether supplying API arguments and return values improves the explanation (RQ2). Here the

judge scores do not support a positive effect: adding parameters significantly *lowers* the total under both judges (with- vs. without-parameter totals: GPT-4o 12.37 vs. 13.22, two-sided Mann–Whitney $p = 1.8 \times 10^{-16}$, Cohen’s $d = 0.58$; GPT-4.1 12.88 vs. 13.96, $p = 1.3 \times 10^{-13}$, $d = 0.61$), with all three criteria declining and the largest decline on Clarity for GPT-4o but on Coverage for GPT-4.1 (Table 13). A plausible reading is that the raw arguments and return values add length and detail that the judges do not credit as substance—charged to Clarity by one judge and to Coverage by the other—and that the family signal is already largely carried by the API names and categories. Accuracy also declines under both judges, so the added context does not merely dilute the presentation: the judges rate the resulting rationales as less accurate as well. This negative result is itself a useful contribution for Gap 2. It is natural to assume that richer, argument-level context can only help an LLM explanation, yet our controlled comparison shows no such benefit under an automated judge—if anything, a small penalty. The *capability* to supply such context is novel, but it must be exercised selectively—summarizing the most informative arguments rather than passing them verbatim—before any benefit is likely to appear. Reporting this spares later work as a tempting but, on this evidence, unproductive design choice. Because the LLM-as-a-Judge protocol rewards plausibility and readability, a Clarity penalty for verbose context is expected; whether argument-level context helps a human analyst remains an open question for the expert evaluation discussed below.

5.4. Per-Family Alignment of Classification, Faithfulness, and Interpretation

Because the three experiments form a chain (Figure 5), it is informative to read their per-family results together (Tables 6, 11 and 14). The three axes—classification recall, Grad-CAM faithfulness, and LLM interpretation quality—do not co-vary cleanly across families. Faithfulness tracks classification only loosely: the two best-classified families, Berbew and Vbclone, also rank highest in faithfulness (0.7981 and 0.3493), and the worst-classified family, Noon, ranks lowest (0.0755), yet Cosmu is classified almost perfectly while its faithfulness is among the lowest (0.1607). Interpretation quality is more closely decoupled still: with the corrected API context, all ten families fall within a narrow band (12.01–13.51/15 for GPT-4o) regardless of their faithfulness or recall, so neither a faithful saliency map nor an accurate prediction reliably predicts how the judge scores the resulting rationale. Berbew is illustrative: it has the most faithful saliency (0.7981) and perfect classification, yet its interpretation score (13.20/15) is unremarkable relative to the other families, so high faithfulness does not translate into a distinctively higher interpretation score. Taken together, these patterns support the paper’s central characterization claim: a family the classifier handles well, and whose saliency is faithful, can still be interpreted poorly, so the three properties must be measured separately rather than inferred from one another.

5.5. Qualitative Case Study

To make the end-to-end behavior concrete, Figure 7 traces a single correctly classified test sample—a Vbclone instance (test index 40)—through the full pipeline. The classifier assigns the correct family with 99.99% confidence; Grad-CAM localizes the salient region of the GAF image, which reverse-maps to ten contributing API calls, among them `GetModuleHandleA`, `LdrGetProcedureAddressForCaller`, and `LdrLoadDll`, together with the native memory-management trio `NtMapViewOfSection`, `NtAllocateVirtualMemory`, and `NtProtectVirtualMemory`. From these calls the LLM produces a structured rationale that links the dynamic library-resolution calls to runtime dependency loading and the memory-management calls to in-memory payload staging, and, under the zero-shot, name-and-category prompt, the automated judges award it a near-perfect total (15/15 from both

judges), whereas in our human expert evaluation the same rationale received an accuracy of only 1 out of 5, because the cited calls were generic to most Windows programs.

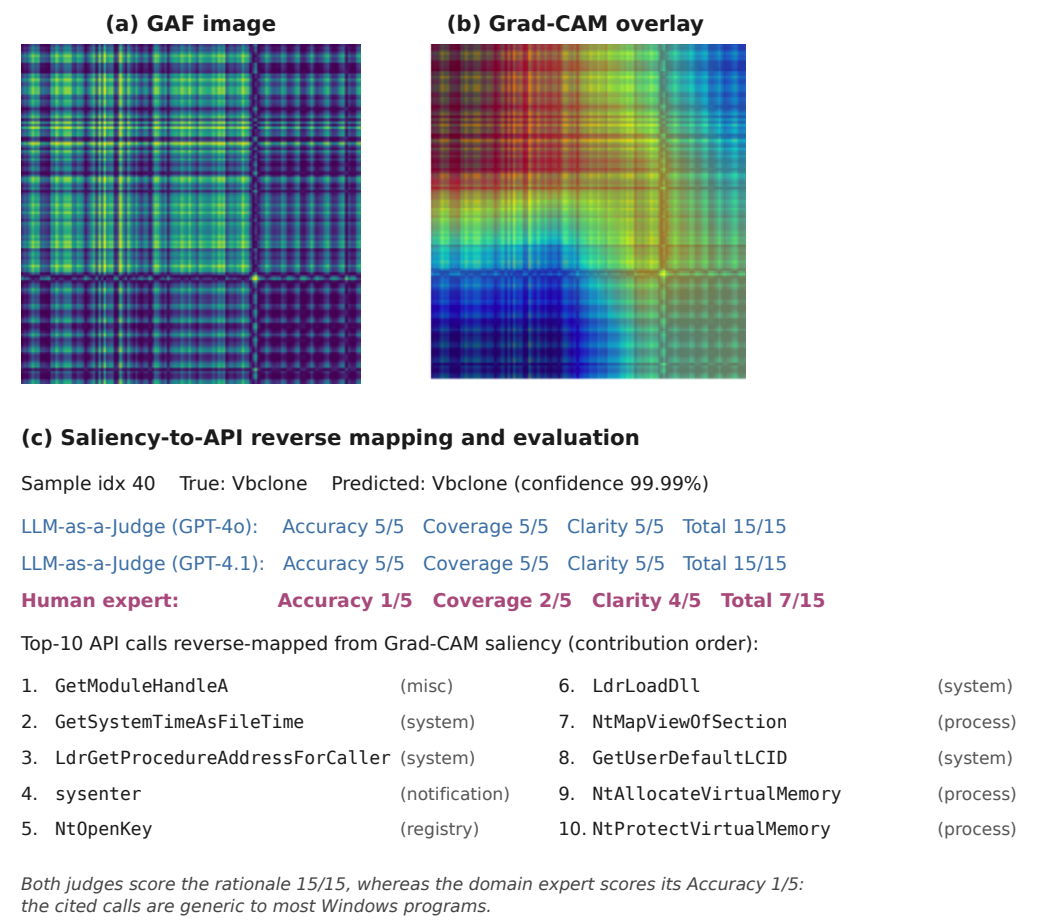


Figure 7. End-to-end case study for a correctly classified Vbclone sample (test index 40, predicted with 99.99% confidence): (a) the GAF image; (b) the Grad-CAM saliency overlay; and (c) the ten API calls reverse-mapped from the salient region, together with the scores that both judges and the human expert assigned to the generated rationale (zero-shot, name-and-category prompt).

This example illustrates both the promise and the principal caution of the pipeline. On the one hand, the components compose into a coherent, fluent, human-readable account that is grounded in the specific calls the classifier relied on, showing that the integration is feasible end-to-end. On the other hand, the rationale attaches confident, family-specific behavioral interpretations—process hollowing, anti-sandbox timing checks, and locale-based target selection—to API calls that are individually generic to most Windows programs, and the reference-free judge, lacking a ground-truth rationale, rewards this fluency with the maximum score. The case thus concretely exhibits the over-attribution tendency and the limits of LLM-as-a-Judge evaluation noted in Section 5.7, and is corroborated by the human expert evaluation of Section 4.3.3.

5.6. Implications

This study establishes three things. First, a pipeline that couples GAF-based classification, Grad-CAM reverse-mapping, and single-agent LLM interpretation runs end-to-end and is fully reproducible in a containerized environment. Second, the three coupled experiments constitute a reference-free framework for evaluating such a pipeline along both axes—classification and explanation—without requiring ground-truth rationales. Third, the experiments yield concrete, transferable findings: order preservation shows no robust

classification advantage over a frequency-only encoding under cross-validation (the two are comparable), Grad-CAM faithfulness is strongly family-dependent, and raw argument-level context significantly reduces LLM-judged explanation quality. On this basis, the pipeline addresses a practical gap in image-based malware analysis: a family label alone offers an analyst little to act on, whereas a rationale that names the specific API calls responsible for a prediction can support triage and review more directly. This practical value is conditional on the rationale being faithful to the classifier (Experiment 2) and judged as adequate (Experiment 3); demonstrating that it is genuinely useful to a human analyst, rather than merely plausible, requires the expert evaluation noted below. We therefore position the pipeline as a feasible, reproducible framework for explainable classification whose component behaviors we have measured, rather than as a demonstration of analyst-level utility.

5.7. Limitations and Future Work

Several limitations bind the present study. First, evaluation is confined to a single dataset (WinMET); although this enables a controlled method comparison, generalization to other sandboxes and label schemes remains to be tested. Second, Experiment 1 centers on a controlled comparison of two encodings and adds a single 1D-CNN sequence-model baseline (Table 8) rather than a broad benchmark against state-of-the-art API-sequence classifiers, so it speaks to the value of order preservation and to parity with one sequence model rather than to absolute competitiveness against the strongest available methods. Third, the GAF image is compressed to 128×128 , which, through the underlying piecewise aggregate approximation, maps each pixel to a short segment of the sequence and may lose fine-grained ordering for very long sequences; the ablation in Table 9 makes the resulting trade-off explicit, since a length of 1000 classifies significantly better than 500 while doubling the number of calls represented by each pixel, so deployments that do not need call-level attribution may prefer a longer sequence. Fourth, Grad-CAM faithfulness measures how well the saliency reflects the *classifier*, not whether the resulting explanation is factually correct about the sample. Fifth, explanation quality is scored by two independent automated judges (GPT-4o and GPT-4.1), which assess the plausibility and internal consistency of a rationale rather than its ground-truth correctness. The rationales—and the few-shot demonstrations—also attribute behavior partly on the basis of family-level prior knowledge rather than solely the listed API calls. We mitigate these two limitations by using two judges rather than one—under which the same conclusions hold—and by adding a small human expert evaluation; that evaluation is itself limited in that it is performed by the authors as domain experts, is small-scale ($n = 40$), and does not measure inter-rater reliability, though scoring is blind to the judge scores under a pre-specified rubric. Sixth, in the few-shot configuration the prompt includes a worked example of the predicted family, which can prime the model toward that family. Seventh, high-saliency faithfulness does not translate into a distinctively higher interpretation score: Berbew has the most faithful saliency yet an unremarkable interpretation total (13.20–13.34). Over-attribution of intent to generic API calls remains a genuine tendency of the rationales, as the human expert evaluation and the Vbclone case study of Section 5.5 show. Eighth, the argument-level context we introduced significantly *reduced* judged explanation quality under two independent judges ($p < 10^{-12}$, Cohen's $d \approx 0.6$), so, on this evidence, passing raw arguments verbatim is counterproductive rather than merely unhelpful. Ninth, GASF normalizes the vocabulary indices and encodes their pairwise magnitude relationships, yet these indices are nominal rather than ordinal; the resulting image geometry therefore depends partly on how indices are assigned. We test this sensitivity by re-running under three permuted assignments (Section 4.2, Table 10): accuracy and AUC-ROC are unchanged, but a residual fluctuation

in Macro-F1/Weighted-F1 remains, driven by the trial-to-trial variance in a minority family (Amadey) rather than by systematic degradation.

Tenth, the five-fold cross-validation is itself limited by its size: with $k = 5$ folds the Wilcoxon signed-rank test cannot by construction fall below $p = 0.0625$, so the encoding comparison is interpreted through the paired effect size and the confidence interval rather than through significance, and a larger number of folds or repeated cross-validation would be needed to test it more sharply.

These limitations map onto the concrete next steps. Comparison against a broader range of sequence-based classifiers and stronger baselines (extending the 1D-CNN comparison added here) would place the order-preservation effect on an absolute scale, and larger-scale human expert evaluation would test whether the explanations are useful rather than merely plausible. To raise and stabilize Grad-CAM faithfulness, finer reverse-mapping (the current resolution maps each pixel to several calls), higher-resolution images, and complementary attribution methods are natural candidates; selecting and summarizing the most informative arguments, instead of passing them verbatim, may unlock the value of argument-level context; stronger class-imbalance handling, such as resampling or focal loss, targets the weak minority-family recall; and constraining the rationale to the reverse-mapped calls may curb the over-attribution that the human evaluation finds to be pervasive, as the Vbclone case study illustrates. Finally, ablation of the per-family demonstration, a no-Grad-CAM baseline that supplies the full API list, cross-referencing against public catalogs such as MalAPI.io and MITRE ATT&CK, and extension to additional datasets and sandboxes would each broaden and strengthen the conclusions.

A natural extension addresses Gap 3: a domain-knowledge-guided LLM-agent loop that proposes successive training configurations from prior results and malware-domain knowledge, aiming to reach competitive Macro-F1 with fewer trials than grid search (RQ4). We outline this Layer-2 capability as future work; its empirical comparison against grid and random search, including the effect of injecting domain knowledge, is left for a subsequent study and can be promoted to a dedicated experimental section once completed.

6. Conclusions

In this paper, we have proposed an explainable deep learning pipeline for Windows malware family classification that not only predicts a family but also explains its decision in natural language. The pipeline transforms API call sequences from the WinMET dataset into order-preserving GAF images, classifies them with a ResNet-50, reverse-maps Grad-CAM activations to the contributing API calls, and feeds their names, categories, arguments, and return values to a single LLM agent that produces an API-grounded rationale. Classification and explanation are evaluated as coupled objectives: Experiment 1 compares GAF against a frequency-only heatmap encoding under identical conditions; Experiment 2 quantifies the faithfulness of the Grad-CAM reverse-mapping across families; and Experiment 3 measures explanation quality with a reference-free LLM-as-a-Judge protocol, isolating the contribution of argument-level context.

Empirically, GAF performed comparably to the heatmap encoding on classification, matching it on overall accuracy and, on Macro-F1 (0.6335 vs. 0.6077), it was higher on a single split but statistically comparable under five-fold cross-validation; per-family Grad-CAM faithfulness ranged from 0.0755 (Noon) to 0.7981 (Berbew); and supplying argument-level context significantly reduced LLM-judged explanation quality under two independent judges, with the best score arising from a zero-shot, name-and-category prompt (13.34/15). Across families, these three axes—classification performance, saliency faithfulness, and interpretation quality—did not co-vary cleanly, so a pipeline of this kind must measure each separately rather than infer one from another. These findings answer the first three

research questions: on a single split order preservation favors the per-family (macro) view of classification, though cross-validation shows the two encodings classify comparably (RQ1); the faithfulness of the Grad-CAM reverse-mapping is strongly family-dependent (RQ2); and a single LLM agent can produce an API-grounded rationale (RQ3), although supplying raw argument-level context significantly reduces its judged quality (RQ2). This study therefore meets its stated objective—to demonstrate a feasible, reproducible, explainable pipeline and to characterize its behavior, rather than to maximize classification accuracy or to surpass prior classifiers. Its principal contributions are the integration of order-preserving image classification with API-grounded LLM interpretation; a controlled comparison that isolates the effect of order preservation; the introduction and quantitative evaluation of WinMET argument-level context for explanation; a reference-free protocol for evaluating both classification and explanation; and a fully containerized, reproducible implementation. Future work will develop the domain-knowledge-guided LLM-agent loop for automated experiment optimization (RQ4) and strengthen the evidence through comparison with a broader range of sequence-based classifiers (beyond the 1D-CNN baseline added here), larger-scale human expert evaluation of the generated explanations, and extension to additional datasets.

Author Contributions: Conceptualization, Y.F. and Y.S.; methodology, Y.F. and Y.S.; software, Y.F.; validation, Y.F.; investigation, Y.F. and Y.S.; resources, M.H. and M.M.; data curation, Y.F.; writing—original draft preparation, Y.F.; writing—review and editing, Y.S. and M.H.; visualization, Y.F.; supervision, M.M.; funding acquisition, Y.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The WinMET dataset analyzed in this study is publicly available on Zenodo (<https://doi.org/10.5281/zenodo.12647555>) under a CC BY-NC 4.0 license. The source code developed for this study is available from the corresponding author upon reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. AV-TEST Institute. Malware Statistics & Trends Report. Available online: <https://www.av-test.org/en/statistics/malware/> (accessed on 16 June 2026).
2. Nataraj, L.; Karthikeyan, S.; Jacob, G.; Manjunath, B.S. Malware Images: Visualization and Automatic Classification. In *Proceedings of the 8th International Symposium on Visualization for Cyber Security (VizSec'11), Pittsburgh, PA, USA, 20 July 2011*; ACM: New York, NY, USA, 2011; pp. 1–7. [\[CrossRef\]](#)
3. Zhang, S.; Gao, M.; Wang, L.; Xu, S.; Shao, W.; Kuang, R. A Malware-Detection Method Using Deep Learning to Fully Extract API Sequence Features. *Electronics* **2025**, *14*, 167. [\[CrossRef\]](#)
4. Ma, C.; Li, Z.; Long, H.; Bilal, A.; Liu, X. A Malware Classification Method Based on Directed API Call Relationships. *PLoS ONE* **2025**, *20*, e0299706. [\[CrossRef\]](#) [\[PubMed\]](#)
5. Zhang, H.; Yun, X.; Deng, X.; Zhong, X. Malware Classification Based on GAF Visualization of Dynamic API Call Sequences. In *Proceedings of the 2022 IEEE 22nd International Conference on Communication Technology (ICCT), Nanjing, China, 11–14 November 2022*; IEEE: New York, NY, USA, 2022; pp. 1873–1877.
6. Wang, Z.; Oates, T. Imaging Time-Series to Improve Classification and Imputation. In *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI), Buenos Aires, Argentina, 25–31 July 2015*; pp. 3939–3945.
7. El Rhayati, O.; Essadeq, H.; El Beqqali, O.; Tairi, H.; Lamrini, M.; Riffi, J. Structured Heatmap Learning for Multi-Family Malware Classification: A Deep and Explainable Approach Using CAPEv2. *J. Cybersecur. Priv.* **2025**, *5*, 72. [\[CrossRef\]](#)
8. Selvaraju, R.R.; Cogswell, M.; Das, A.; Vedantam, R.; Parikh, D.; Batra, D. Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV), Venice, Italy, 22–29 October 2017*; pp. 618–626. [\[CrossRef\]](#)

9. Nazim, S.; Alam, M.M.; Rizvi, S.S.; Mustapha, J.C.; Hussain, S.S.; Suud, M.M. Advancing Malware Imagery Classification with Explainable Deep Learning: A State-of-the-Art Approach Using SHAP, LIME and Grad-CAM. *PLoS ONE* **2025**, *20*, e0318542. [[CrossRef](#)] [[PubMed](#)]
10. Liu, S.; Gao, C.; Li, Y. AgentHPO: Large Language Model Agent for Hyper-Parameter Optimization. In Proceedings of the Conference on Parsimony and Learning (CPAL), Stanford, CA, USA, 24–27 March 2025; pp. 1146–1169.
11. Raducu, R.; Villagrasa-Labrador, A.; Rodríguez, R.J.; Álvarez, P. A Dataset of Windows Malware Execution Traces. *Data Brief* **2025**, *63*, 112273. [[CrossRef](#)] [[PubMed](#)]
12. Raducu, R.; Villagrasa-Labrador, A.; Rodríguez, R.J.; Álvarez, P. *WinMET Dataset (Windows Malware Execution Traces)*; Zenodo: Geneva, Switzerland, 2025; CC BY-NC 4.0. [[CrossRef](#)]
13. Wan, L. Enhancing Imbalanced Malware Detection via CWGAN-GP-Based Data Augmentation and TextCNN–Transformer Integration. *Symmetry* **2025**, *17*, 2153. [[CrossRef](#)]
14. Uysal, D.T.; Yoo, P.D.; Taha, K.; Yeun, C.Y.; Damiani, E. A Multi-Label Visualisation Approach for Malware Behaviour Analysis. *Sci. Rep.* **2025**, *15*, 37979. [[CrossRef](#)] [[PubMed](#)]
15. Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E.P.; et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS 2023), New Orleans, LA, USA, 10–16 December 2023.
16. Raducu, R.; Villagrasa-Labrador, A.; Rodríguez, R.J.; Álvarez, P. MALVADA: A Framework for Generating Datasets of Malware Execution Traces. *SoftwareX* **2025**, *30*, 102082. [[CrossRef](#)]
17. He, K.; Zhang, X.; Ren, S.; Sun, J. Deep Residual Learning for Image Recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 27–30 June 2016; pp. 770–778. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.